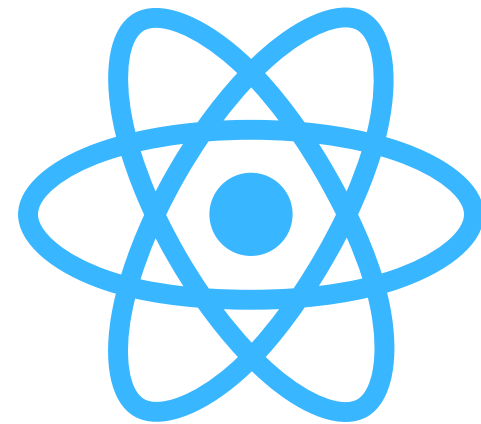




Diginova

Formation

INTRODUCTION À REACT



Formatrice Hanane SADEG

INTRODUCTION

Introduction à React

À la fin de ce chapitre, les étudiants étudiants sauront :

- *Ce qu'est un framework et une bibliothèque JavaScript*
- *Pourquoi React est utilisé massivement en entreprise*
- *Comment fonctionne React (components, props, state...)*
- *La différence entre React, Vite, Node.js, npm, JSX*
- *Le rôle d'un bundler, d'un runtime et d'un framework complet*
- *Les alternatives (Vue, Angular, Next.js...)*

QU'EST-CE QU'UN FRAMEWORK ? QU'EST-CE QU'UNE BIBLIOTHÈQUE ?

Framework (Cadre de travail).

Un framework est un ensemble d'outils + règles + structures qui impose une manière de développer.

🔧 Il fournit :

- une architecture
- des conventions
- des outils intégrés
- un style de développement
- souvent un routeur, un moteur de templates, un système d'injection...

➡ Exemple de frameworks :

Angular, Django, Laravel, Symfony, ASP.NET, Next.js

Le framework dit :

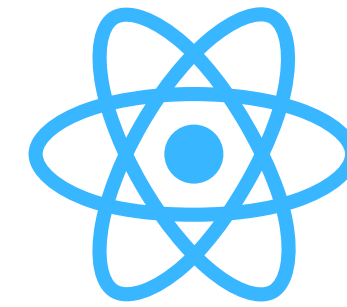
👉 “Voici comment tu dois organiser ton projet et ton code.”

Bibliothèque (Library).

Une bibliothèque est un ensemble de fonctions réutilisables... mais c'est vous qui décidez comment les utiliser.

➡ Exemple :

React, jQuery, Lodash



🧠 **React** n'est PAS un framework, c'est une bibliothèque pour construire des interfaces.

La bibliothèque dit :

👉 “Voici les outils. Organise ton projet comme tu veux.”

POURQUOI UTILISER REACT ?

1. Parce que les interfaces modernes sont complexes

Aujourd'hui une interface doit :

- se mettre à jour sans recharger la page
- gérer l'interactivité en temps réel
- afficher beaucoup de données
- être réactive et fluide
- fonctionner sur mobile, tablette, PC

💡 Les sites modernes ressemblent plus à des applications qu'à des pages HTML statiques.

3. Parce que React est LA technologie la plus demandée

Toutes les grandes entreprises l'utilisent :

- Meta
- Netflix
- Airbnb
- Amazon
- Spotify
- Uber
- PayPal
- Et la majorité des startups

📈 Sur le marché du travail, React = TOP 1 des technologies front-end demandées en Europe.

2. Parce que React simplifie la mise à jour de la page

Avant → DOM manipulé à la main (long, compliqué, risqué)

Maintenant → React gère le DOM pour nous grâce au :

🧠 Virtual DOM

C'est une version légère de l'interface en mémoire, que React compare avec la version réelle pour mettre à jour seulement ce qui change.

➡ Résultat :

- ✓ Performances améliorées
- ✓ Moins d'erreurs
- ✓ Développement plus rapide

4. Parce qu'on peut créer tout type de projets

Avec React, on peut développer :

- des sites vitrines
- des dashboards
- des interfaces de plateformes
- des applications mobiles (React Native)
- des applications desktop (Electron + React)
- des applications SEO-first (Next.js)

REACT, VITE, NODE.JS, NPM : COMMENT ÇA S'ARTICULE ?

🧱 React = la bibliothèque UI

➡ Permet de créer des composants, gérer un état, afficher du JSX.

⚡ Vite = l'outil de développement + bundler

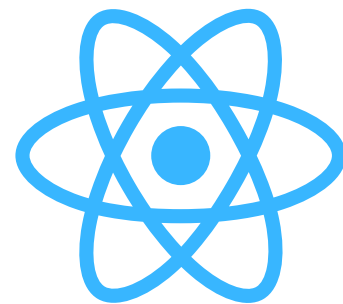
Avant, React utilisait Create-React-App (CRA) → lent, lourd

Aujourd'hui → Vite est devenu le standard

Vite sert à :

- lancer un serveur local ultra rapide
- compiler le JSX
- optimiser le code
- regrouper tous les fichiers JS pour la production
- gérer le Hot Reload (mise à jour automatique)

➡ Vite est un outil, pas un framework.



■ Node.js = environnement d'exécution JavaScript

Node permet :

- d'utiliser JavaScript en dehors du navigateur
- d'exécuter des outils (Vite, npm)

Aujourd'hui, installer Node est obligatoire pour React.



📦 npm / yarn / pnpm = gestionnaires de packages

npm permet :

- d'installer React
- d'installer les dépendances
- de lancer les commandes comme :

```
npm run dev
```



🧬 JSX = JavaScript + XML

React ne manipule pas du HTML mais du JSX, un langage hybride :

```
<h1>Hello</h1>
```

➡ Ce n'est pas du HTML

➡ Ce n'est pas du JS

➡ C'est un mélange ultra pratique

Vite + Babel transforment ce JSX en JS compréhensible par le navigateur.



CONCEPTS FONDAMENTAUX DE REACT

1. Composants

Un composant = une petite partie réutilisable de l'interface.

Exemples :

- navbar
- bouton
- carte produit
- formulaire
- tableau de bord

Chaque composant :

- possède son propre code
- peut recevoir des données (props)
- peut stocker un état interne (state)
- peut être réutilisé partout

2. Props

Les props sont les “paramètres” d'un composant.

3. State

L'état représente :

- les données qui changent
- ce qui doit être réaffiché

Exemples :

- compteur
- input
- liste de tâches
- panier e-commerce
- session utilisateur (connecté / déconnecté)

4. Cycle de vie

Avec React Hooks :

- useEffect() remplace les anciens lifecycle methods
- permet d'appeler une API, écouter des événements...

5. SPA – Single Page Application

Un site React est généralement une SPA :

- Une seule page HTML
- Le contenu change sans recharger la page
- Expérience fluide comme une application

REACT N'EST PAS SEUL : LES FRAMEWORKS AUTOUR

React est une bibliothèque.

Pour faire une application complète, on ajoute des briques :

- ◆ *React Router → système de pages*
- ◆ *Redux / Zustand → gestion de l'état global*
- ◆ *Axios / Fetch → appels API*
- ◆ *Tailwind / Material UI → design*
- ◆ *Next.js → framework complet basé sur React*
- ◆ *React Native → mobile*

POURQUOI LES ENTREPRISES CHOISISSENT REACT ?

✓ *Grande communauté*

Millions de développeurs → facile recrutement

✓ *Performance*

Virtual DOM, composants réutilisables

✓ *Productivité*

Écosystème énorme

✓ *Polyvalence*

Web, mobile, desktop

✓ *Courbe d'apprentissage rapide*

CONCLUSION : POURQUOI VOUS DEVEZ APPRENDRE REACT

- *C'est la compétence la plus demandée pour un développeur front-end*
- *Vous pourrez créer des interfaces modernes*
- *Les stages et alternances recherchent massivement React*
- *C'est la porte d'entrée vers React Native, Next.js, le Cloud, l'IA...*

CHAPITRE 0

Installation & Mise en place de l'environnement React

À la fin de ce chapitre, l'étudiant sera capable de :

- Installer Node.js et vérifier son installation*
- Installer l'éditeur VS Code*
- Installer les extensions utiles pour coder en React*
- Créer un nouveau projet React (méthode moderne avec Vite)*
- Comprendre où écrire du code dans le projet*
- Lancer et afficher son projet dans un navigateur*
- Comprendre la structure du dossier d'un projet React*

INSTALLATION DE NODE.JS

Pourquoi Node.js est nécessaire ?

React utilise Node.js pour :

- télécharger les dépendances (npm)
- exécuter les commandes React
- lancer le serveur de développement
- compiler automatiquement le code

Sans Node.js → impossible de démarrer un projet React.

▶ Étapes d'installation

1. Aller sur : 🖱️ <https://nodejs.org>

2. Télécharger la version :

💡 LTS (Long Term Support) → la plus stable

3. Installer (clics classiques : Suivant → Suivant → Terminer)

▶ Vérification de l'installation

Ouvrir un terminal (ou PowerShell) et taper :

```
node -v  
npm -v
```

Si les deux affichent un numéro → Node.js est installé correctement ✓



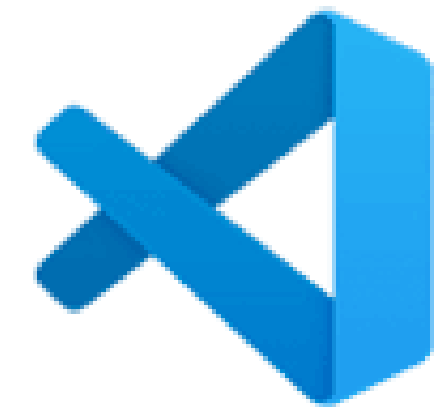
INSTALLER VISUAL STUDIO CODE (VS CODE)

Pourquoi VS Code ?

- *Léger, rapide, gratuit*
- *Parfait pour React*
- *Extensions puissantes*
- *Auto-complétion avancée*
- *Très utilisé en entreprise*

▶ *Installation*

1. *Télécharger : <https://code.visualstudio.com/>*
2. *Installer comme un logiciel normal*
3. *Lancer VS Code*



Visual Studio Code

EXTENSIONS INDISPENSABLES POUR REACT

Ouvrir VS Code →

Extensions (Ctrl + Shift + X) → taper :

- ES7+ React/Redux/React-Native snippets
→ crée automatiquement des composants React
(Ex : taper rafce → il génère un composant complet)
- Prettier – Code formatter
→ met automatiquement le code en forme
- Material Icon Theme
→ icônes de fichiers visuelles (meilleure lisibilité)

CRÉER UN PROJET REACT MODERNE AVEC VITE

Pourquoi Vite ?

- Plus rapide
- Plus moderne
- Temps de démarrage quasi instantané
- Rechargement ultra rapide
- Recommandé en 2025+

▶ Commandes pour créer un projet

Dans un dossier de travail :

```
npm create vite@latest my-react-app --template react
```

Puis :

```
cd my-react-app  
npm install
```

▶ Alternative classique : Create React App
(À utiliser si l'école a une infra ancienne)

```
npx create-react-app my-react-app  
cd my-react-app  
npm start
```

▶ Ce que Vite a généré
Une arborescence simple :

```
my-react-app  
├── src → ton code React  
│   ├── App.jsx  
│   └── main.jsx  
├── public → fichiers statiques  
├── package.json → dépendances & scripts  
├── vite.config.js → config interne  
└── node_modules → dépendances (ne pas toucher)
```



Vite + React

LANCER LE PROJET REACT

Depuis le dossier du projet :

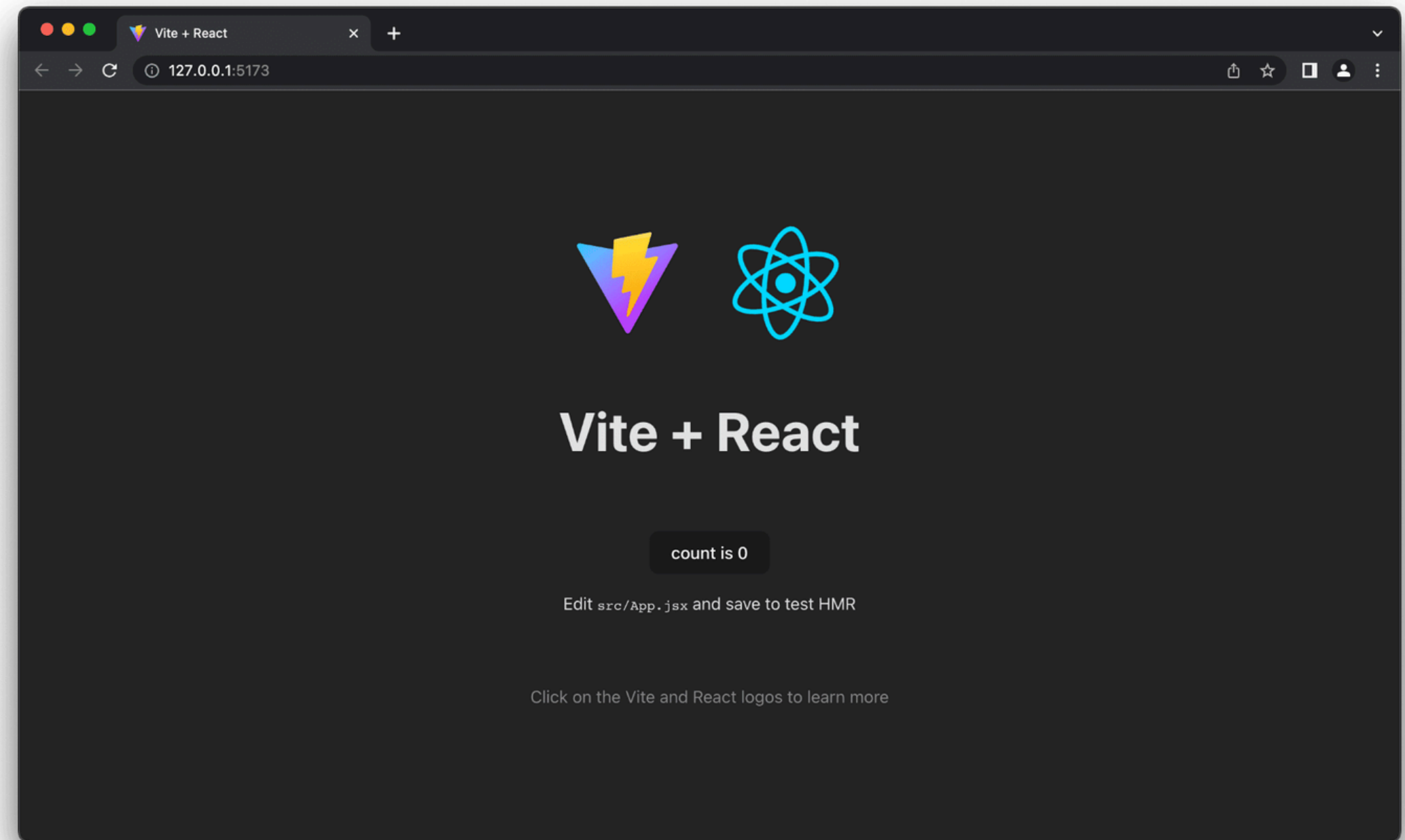
```
npm run dev
```

Le terminal affiche :

```
Local: http://localhost:5173
```

➡ Ouvre le lien dans ton navigateur.

Ton premier projet React fonctionne !



COMPRENDRE LA STRUCTURE DU DOSSIER REACT

 `src/`

Contient tout ton code React :

- `App.jsx` → composant principal
- `main.jsx` → point d'entrée
- composants → à créer ici

 `App.jsx`

```
function App() {  
  return <h1>Hello React !</h1>;  
}  
  
export default App;
```

C'est la première interface affichée.

 `main.jsx`

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";  
  
ReactDOM.createRoot(document.getElementById("root")).render(  
  <App />  
);
```


PREMIER TEST : MODIFIER APP.JSX

Remplacer dans votre code :

```
<h1>Hello React !</h1>
```

Par :

```
<h1>Bienvenue dans mon premier projet React</h1>
```

- ➡ *La page se met à jour automatiquement*
- ➡ *C'est le HMR (Hot Module Reloading)*

CHAPITRE 1

BASES DE REACT

Objectifs

- *Comprendre à quoi sert JavaScript*
- *Savoir exécuter du JS dans la console*
- *Comprendre les variables et types*

COMPOSANTS

Définition

Un composant est une petite brique d'interface réutilisable.
Il peut représenter un bouton, un titre, une carte produit, un bloc d'une page...

Exemple simple :

```
function Titre() {  
  return <h1>Bienvenue dans React</h1>;  
}
```

Idée clé

Chaque composant peut être réutilisé plusieurs fois comme un Lego.

PROPS

Définition

Les props sont des données envoyées à un composant pour le personnaliser.

Exemple :

```
function Bonjour(props) {  
  return <h1>Bonjour {props.nom} !</h1>;  
}
```

// Utilisation

```
<Bonjour nom="Sarah" />
```

Rôle

Personnaliser un composant pour afficher des valeurs différentes.

STATE

Définition

Le state représente une donnée interne au composant, qui peut évoluer (comme un compteur, une valeur d'input...).

Exemple :

```
const [count, setCount] = useState(0);
```

Rôle

Permet de rendre une interface interactive et dynamique.

JSX

Définition

JSX est un langage qui mélange du HTML dans du JavaScript.

Exemple :

```
return (  
  <div>  
    <h1>Hello React</h1>  
  </div>  
);
```

Caractéristiques

- ressembler à du HTML
- permet d'insérer du JS dans { }
- un seul élément parent doit être retourné

USESTATE

Définition

useState est un Hook qui permet de créer et gérer des états dans un composant.

Exemple :

```
function Compteur() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <h1>Compteur : {count}</h1>  
      <button onClick={() => setCount(count + 1)}>  
        Ajouter  
      </button>  
    </>  
  );  
}
```

EXCERCICE

Exercice 1 – Composant “Titre”

Créer un composant Titre qui affiche :
"Bienvenue dans React"

Exercice 2 – Afficher la date du jour

Créer un composant DateAujourd'hui qui affiche la date via :

```
new Date().toLocaleDateString()
```


EXCERCICE - CORRECTION

Exercice 1 : Composant “Titre”

💡 Objectif : afficher un simple texte

```
function Titre() {  
  return <h1>Bienvenue dans React</h1>;  
}  
  
export default Titre;
```

📌 À retenir

Un composant React est une fonction qui retourne du JSX.

Le texte affiché doit être entouré d'un seul élément (ici <h1>).

EXCERCICE - CORRECTION

Exercice 2 : Composant “DateAujourd’hui”

💡 Objectif : afficher la date actuelle

```
function DateAujourd’hui() {  
  const date = new Date().toLocaleDateString();  
  
  return <p>Date du jour : {date}</p>;  
}  
  
export default DateAujourd’hui;
```

📌 À retenir

- `new Date()` crée un objet date JavaScript
- `.toLocaleDateString()` formate la date pour l'utilisateur
- Les accolades `{ }` permettent d'insérer du JavaScript dans du JSX

CHAPITRE 2

STATE & ÉVÉNEMENTS

À la fin de ce chapitre, l'étudiant sera capable de :

- *Comprendre le concept de state dans React*
- *Utiliser le Hook `useState()`*
- *Gérer les événements utilisateur*
- *Créer des composants interactifs*
- *Construire un compteur interactif*
- *Raffiner l'expérience utilisateur*

LE STATE

Qu'est-ce que le state ?

- *Une information interne au composant*
- *Qui peut changer au cours du temps*
- *Et qui met automatiquement à jour l'interface*

Exemple :

```
const [count, setCount] = useState(0);
```

GESTION DES ÉVÉNEMENTS

React permet d'écouter des actions :

- clic (`onClick`)
- saisie (`onChange`)
- survol (`onMouseEnter`)
- etc.

Exemple :

```
<button onClick={() => alert("Clic !")}>  
  Cliquez ici  
</button>
```

```
function Compteur() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>Compteur : {count}</p>  
      <button onClick={() => setCount(count + 1)}>  
        Ajouter  
      </button>  
    </div>  
  );  
}
```


EXCERCICE

Exercice 1 – Le compteur

Créer un composant avec deux boutons :

+ augmenter

– diminuer

Exercice 2 – Saisie en direct

Un input + un <p> qui affiche en temps réel ce que l'utilisateur tape.

Utiliser :

```
const [texte, setTexte] = useState("");
```

EXCERCICE - CORRECTION

Exercice 1

```
import { useState } from "react";

function Compteur() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Compteur : {count}</h2>

      <button onClick={() => setCount(count + 1)}>⬆ Augmenter</button>
      <button onClick={() => setCount(count - 1)}>⬇ Diminuer</button>
    </div>
  );
}

export default Compteur;
```



À retenir

- `useState(0)` crée une variable `count` et une fonction `setCount`
- Modifier le state provoque une mise à jour automatique de l'interface
- Chaque clic déclenche une fonction qui met à jour le state

EXCERCICE - CORRECTION

Exercice 2

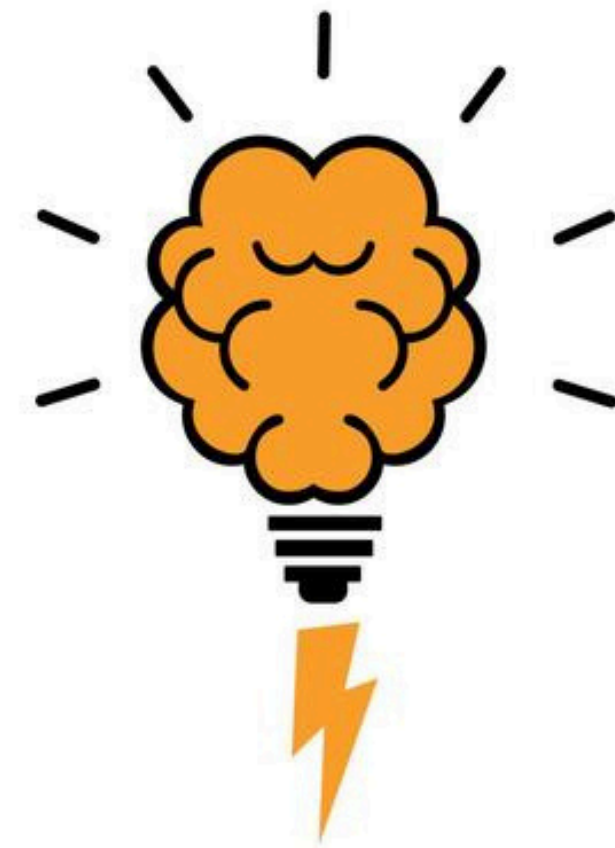
```
import { useState } from "react";

function SaisieDirecte() {
  const [texte, setTexte] = useState("");

  return (
    <div>
      <input
        type="text"
        placeholder="Tape quelque chose..."
        onChange={(e) => setTexte(e.target.value)}
      />

      <p>Vous avez tapé : {texte}</p>
    </div>
  );
}

export default SaisieDirecte;
```



À retenir

- `onChange` se déclenche à chaque frappe du clavier
- `e.target.value` contient le texte tapé dans l'input
- `texte` est mis à jour en direct : React réaffiche le composant automatiquement

CHAPITRE 3

LISTES & PROPS

À la fin de ce chapitre, l'étudiant sera capable de :

- *Afficher une liste d'éléments en React*
- *Utiliser les props pour passer des données à un composant*
- *Créer des composants réutilisables pour afficher des données*
- *Manipuler des tableaux de données*
- *Comprendre le flux de données descendant en React (one-way data flow)*

LES LISTES EN REACT

Pour afficher une liste, on utilise .map().

Exemple :

```
const fruits = ["Pomme", "Banane", "Kiwi"];

return (
  <ul>
    {fruits.map((fruit) => (
      <li>{fruit}</li>
    ))}
  </ul>
);
```

PROPS + LISTES : CARDPRODUIT

Exemple d'objet :

```
const produit = {  
  nom: "Laptop",  
  prix: 799  
};
```

Composant :

```
function CardProduit(props) {  
  return (  
    <div className="card">  
      <h3>{props.nom}</h3>  
      <p>{props.prix} €</p>  
    </div>  
  );  
}
```

Afficher plusieurs produits :

```
const produits = [  
  { nom: "Clavier", prix: 29 },  
  { nom: "Souris", prix: 19 },  
  { nom: "Écran", prix: 199 }  
];  
  
return (  
  <div>  
    {produits.map((p) => (  
      <CardProduit nom={p.nom} prix={p.prix} />  
    ))}  
  </div>  
);
```

EXERCICES

Exercice 1 – Liste de produits

Créer un tableau d'objets produits (nom, prix)
→ l'afficher avec `.map()`

Exercice 2 – Composant CardProduit

Créer un composant réutilisable pour afficher un produit.

EXERCICES - CORRECTION

Exercice 1

```
function ListeProduits() {  
  const produits = [  
    { nom: "Clavier", prix: 29 },  
    { nom: "Souris", prix: 19 },  
    { nom: "Écran", prix: 199 },  
    { nom: "Casque", prix: 49 }  
  ];  
  
  return (  
    <div>  
      <h2>Liste des produits</h2>  
  
      <ul>  
        {produits.map((prod, index) => (  
          <li key={index}>  
            {prod.nom} – {prod.prix} €  
          </li>  
        ))}  
      </ul>  
    </div>  
  );  
}
```

```
export default ListeProduits;
```

À retenir

- `.map()` permet de transformer un tableau → une liste JSX
- Chaque élément doit avoir une key (ici index)
- JSX permet d'insérer JS avec `{ }`

EXERCICES - CORRECTION

```
function CardProduit({ nom, prix }) {  
  return (  
    <div style={{  
      border: "1px solid #ccc",  
      padding: "10px",  
      margin: "10px",  
      borderRadius: "8px"  
    }}>  
      <h3>{nom}</h3>  
      <p>Prix : {prix} €</p>  
    </div>  
  );  
}  
  
export default CardProduit;
```

À retenir

- Les props permettent de rendre un composant flexible
- Un composant peut être utilisé plusieurs fois avec des données différentes
- `.map()` est indispensable pour afficher des listes en React

LA NAVIGATION ENTRE LES PAGES EN REACT

Pourquoi la navigation est différente en React ?

Dans un site classique (HTML), changer de page = charger un nouveau fichier .html.

➡ En React, nous utilisons une Single Page Application (SPA).

Il n'y a qu'une seule page HTML (index.html), mais React change le contenu affiché dynamiquement.

👉 Résultat :

- Navigation ultra rapide
- Pas de rechargement de page
- Expérience fluide (comme une app mobile)

Pour gérer cette navigation, on utilise une bibliothèque : **React Router**

Installer React Router

Dans le terminal du projet React (Vite ou Create React App) :

```
npm install react-router-dom
```

Cette bibliothèque permet :

- de définir des routes
- de dire quelle page afficher pour quelle URL
- de naviguer sans rechargement
- de passer des paramètres entre pages

Les Routes : Comment ça marche ?

Une route associe :

- une URL → /page1
- un composant React → <Page1 />

Exemple simple :

```
<Route path="/" element={<Home />} />
<Route path="/page2" element={<Page2 />} />
```

Configuration de base

Dans main.jsx (ou index.js) on entoure l'application avec :

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Page1 />} />
    <Route path="/page2" element={<Page2 />} />
  </Routes>
</BrowserRouter>
```

👉 BrowserRouter : active la navigation

👉 Routes : conteneur des routes

👉 Route : définit quelle page afficher

LA NAVIGATION ENTRE LES PAGES EN REACT

Naviguer entre les pages avec <Link>

Pour basculer d'une page à l'autre sans recharger :

```
import { Link } from "react-router-dom";

<Link to="/page2">Aller à la Page 2</Link>
```

➡ C'est l'équivalent React du `<a href="">`, mais sans recharger la page.

Bonne pratique : toujours utiliser `<Link>`.

Naviguer depuis un bouton (programmation)

En React, on peut aussi naviguer depuis un événement JavaScript, par exemple quand :

- on clique sur un bouton
- on envoie un formulaire
- on effectue une action

On utilise alors le **hook** `useNavigate()` :

```
import { useNavigate } from "react-router-dom";

const navigate = useNavigate();

function allerVersPage2() {
  navigate("/page2");
}
```

```
<button onClick={allerVersPage2}>Cliquer pour aller à Page 2</button>
```

Structure recommandée pour un projet React moderne

```
src/
├── pages/
│   ├── Home.jsx
│   ├── Login.jsx
│   ├── Dashboard.jsx
│   └── ProjectDetails.jsx
├── components/
│   ├── Navbar.jsx
│   └── Footer.jsx
├── App.jsx
└── main.jsx
```

👉 Chaque page = un composant React

👉 App.jsx = structure globale

👉 main.jsx = définition des routes

LA NAVIGATION ENTRE LES PAGES EN REACT

Exemple simple complet

📄 main.jsx

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Page1 />} />
    <Route path="/page2" element={<Page2 />} />
  </Routes>
</BrowserRouter>
```

📄 Page1.jsx

```
import { Link } from "react-router-dom";

export default function Page1() {
  return (
    <div>
      <h1>Page 1</h1>
      <Link to="/page2">Aller à Page 2</Link>
    </div>
  );
}
```

📄 Page2.jsx

```
import { Link } from "react-router-dom";

export default function Page2() {
  return (
    <div>
      <h1>Page 2</h1>
      <Link to="/">Retour à Page 1</Link>
    </div>
  );
}
```

Navigation avec paramètres (bonus).

Pour afficher un projet / utilisateur / produit :

URL → /project/12

Définir une route paramétrée :

```
<Route path="/project/:id" element={<ProjectDetails />} />
```

Dans la page :

```
import { useParams } from "react-router-dom";

const { id } = useParams();
```

CHAPITRE 4

STATE, FORMULAIRES, LISTES, CRUD LOCAL

À la fin, vous saurez :

- Créer et gérer un state avec `useState`
- Réagir aux actions utilisateur (events)
- Construire des formulaires contrôlés
- Manipuler des listes dynamiques (`map`, `filter`)
- Faire un CRUD complet côté front (Create/Read/Update/Delete)

USESTATE

useState : syntaxe

```
const [value, setValue] = useState(initialValue);
```

Problème : une variable change, mais l'UI ne se met pas à jour

Solution : useState déclenche un re-render

Exemple minimal : compteur

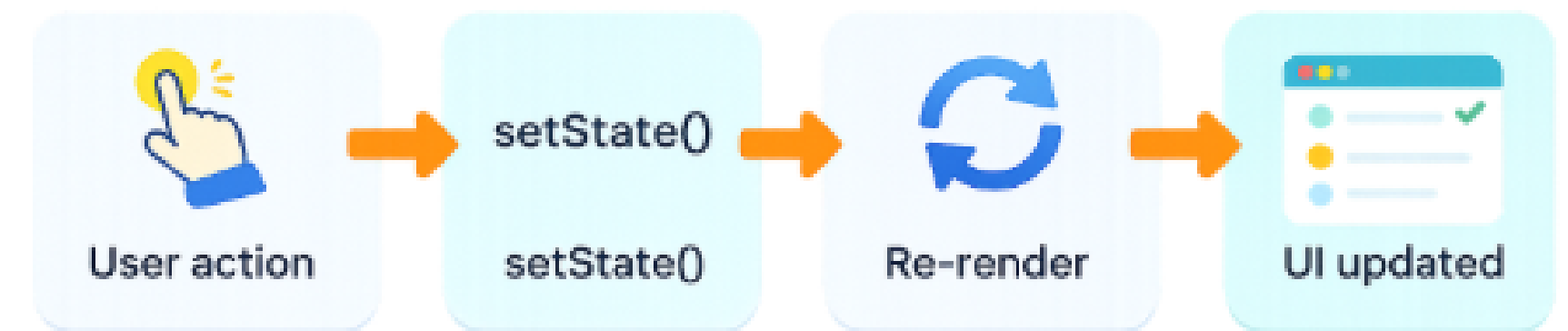
```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

value = état actuel

setValue() = mise à jour (déclenche un re-render)



Variante recommandée :

```
setCount(prev => prev + 1);
```

Pourquoi ? "quand la valeur dépend de l'ancienne"

A retenir

- ✓ *value = lecture*
- ✓ *setValue = écriture + re-render*
- ✓ *useState = mémoire locale du composant*

USESTATE - UTILISATION DE PREV

⚠ **Règle d'or** : si ça dépend de l'ancien state, utilise `prev`

Juste en 1 ligne :

Exemple : 2 clics très rapides sur "+1" → `prev` évite les incohérences.

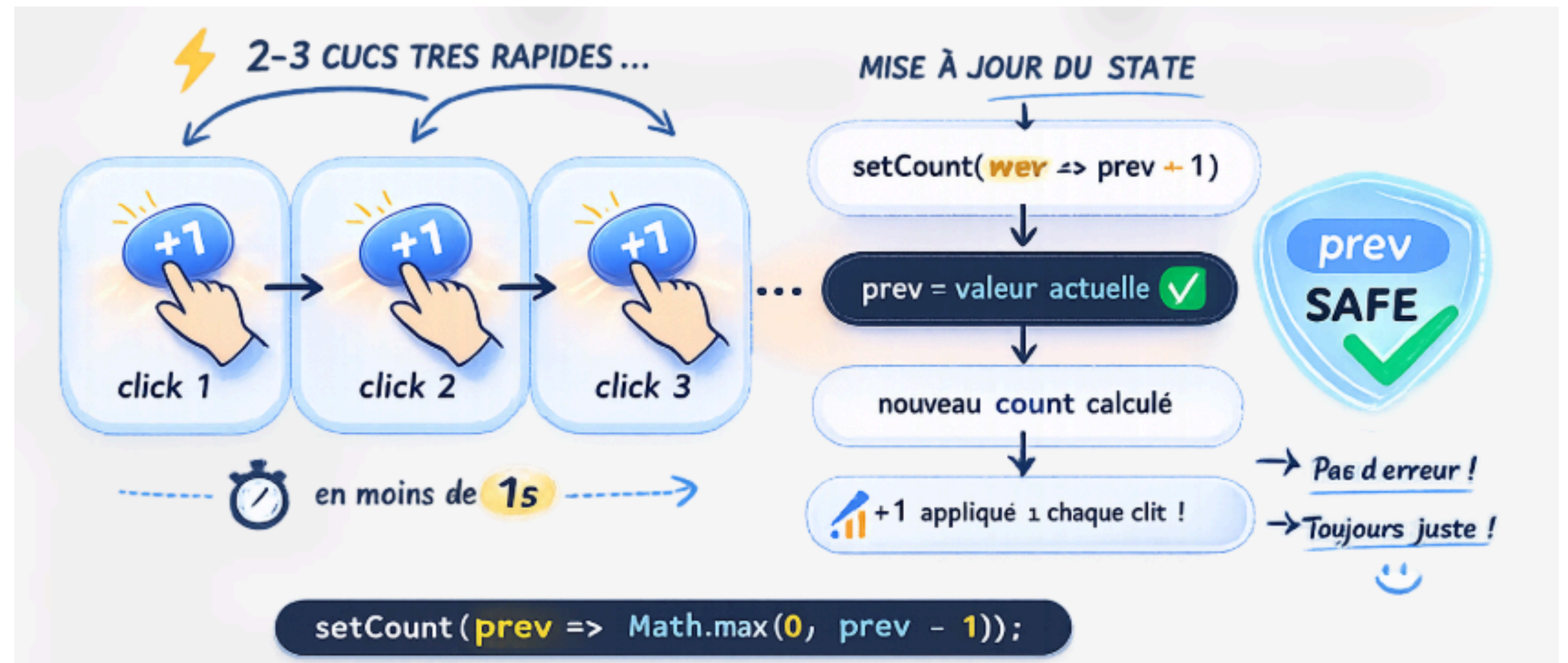
Mini-exo (à faire)

Bouton -1 (min 0)

Bouton +5

✅ **Correction**

```
<button onClick={() => setCount(prev => Math.max(0, prev - 1))}>-1</button>
<button onClick={() => setCount(prev => prev + 5)}>+5</button>
```



👉 Ça fait pratiquer `prev` 2 fois.

USESTATE AVEC UN OBJET (IMMUABILITÉ)

Mettre à jour un seul champ d'un objet dans le state sans casser React.

Exemple : State = objet

```
const [user, setUser] = useState({
  name: "",
  email: ""
});
```

Règle d'or : on ne modifie jamais l'objet directement

✗ Mauvais (mutation)

```
user.name = "Ali";
```

✓ Bon (copie + mise à jour)

```
setUser({ ...user, name: "Ali" });
```

Cas réel : input contrôlé (formulaire)

```
<input
  value={user.name}
  onChange={(e) => setUser({ ...user, name: e.target.value })}
/>

<input
  value={user.email}
  onChange={(e) => setUser({ ...user, email: e.target.value })}
/>
```

★ Version “pro” : une fonction générique (multi-champs)

```
const handleChange = (e) => {
  setUser({ ...user, [e.target.name]: e.target.value });
};
```

```
<input name="name" value={user.name} onChange={handleChange} />
<input name="email" value={user.email} onChange={handleChange} />
```

⚠ Erreurs fréquentes (à afficher en bas)

✗ Oublier ...user → tu perds les autres champs

✗ Modifier user.name = ... → React ne détecte pas correctement le changement

✓ Toujours : copier puis modifier

EVENTS & INTERACTION

Events React : `onClick`, `onChange`, `onSubmit`

Exemple :

```
<button onClick={handleClick}>Valider</button>
```

Passer un paramètre

✓ Correct :

```
<button onClick={() => deleteItem(id)}>Supprimer</button>
```

✗ Incorrect :

```
<button onClick={deleteItem(id)}>Supprimer</button>
```

(Car exécuté immédiatement)

Exo :

Au clic, changer un texte “ON/OFF” (toggle)

✓ Correction :

```
const [isOn, setIsOn] = useState(false);

<button onClick={() => setIsOn(!isOn)}>
  {isOn ? "ON" : "OFF"}
</button>
```

FORMULAIRES CONTRÔLÉS

Controlled input (concept)

Un input contrôlé = sa valeur vient du state.

```
const [name, setName] = useState("");

<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

Formulaire multi-champs

```
const [form, setForm] = useState({
  name: "",
  email: "",
  message: "",
});
```

Mise à jour générique via name=

```
<input name="name" value={form.name} onChange={handleChange} />
<input name="email" value={form.email} onChange={handleChange} />

const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
};
```

Soumission formulaire

```
const handleSubmit = (e) => {
  e.preventDefault();
  console.log(form);
};
```

Validation simple

```
if (!form.email.includes("@")) {
  setError("Email invalide");
  return;
}
```

FORMULAIRES CONTRÔLÉS

Exo : vérifier

name $\geq$ 2 caractères

email valide (regex simple)

message $\geq$ 10 caractères

Correction :

```
const emailOk = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email);
if (form.name.trim().length < 2) return "Nom trop court";
if (!emailOk) return "Email invalide";
if (form.message.trim().length < 10) return "Message trop court";
return "";
```


LISTES DYNAMIQUES

Afficher une liste avec map()

```
items.map(item => <li key={item.id}>{item.name}</li>);
```

Pourquoi key ?

React utilise key pour identifier chaque élément.

✓ key stable : id

✗ index (risque de bugs si suppression / tri)

Filtrer une liste

```
const filtered = items.filter(i => i.name.includes(search));
```

LISTES DYNAMIQUES

Exo :

barre de recherche (filtre live)

✓ Correction :

```
const [search, setSearch] = useState("");

const filtered = items.filter(i =>
  i.name.toLowerCase().includes(search.toLowerCase())
);


```

CRUD LOCAL COMPLET

CRUD : définition { Create (ajouter), Read (afficher), Update (modifier), Delete (supprimer)}

Create (ajouter)

```
setItems([...items, { id: Date.now(), name: form.name }]);
```

Delete

```
setItems(items.filter(i => i.id !== id));
```

Update

```
setItems(items.map(i => i.id === form.id ? form : i));
```

Pattern “édition”

Cliquer “modifier” → remplir le formulaire avec l’item

Au submit → update

Exemple :

```
const addContact = (c) => setContacts([...contacts, { ...c, id: Date.now() }]);  
const deleteContact = (id) => setContacts(contacts.filter(c => c.id !== id));  
const updateContact = (c) => setContacts(contacts.map(x => x.id === c.id ? c : x));
```

USEEFFECT (BASES + PIÈGES)

Objectifs

Comprendre useEffect

Savoir quand il s'exécute

Éviter les boucles infinies

Charger des données au montage

useEffect : définition

Sert à gérer un “effet secondaire”

Exemple : charger des données, écouter un event, localStorage, timer...

Syntaxe

```
useEffect(() => {  
  // effet  
}, [dependencies]);
```

Exemple : charger des données au montage

```
const [items, setItems] = useState([]);  
  
useEffect(() => {  
  setItems(["React", "Node", "DevOps"]);  
}, []);
```

USEEFFECT (BASES + PIÈGES)

Piège : boucle infinie



```
useEffect(() => {  
  setItems([...items, "X"]);  
});
```



```
useEffect(() => {  
  setItems(["X"]);  
}, []);
```


USEEFFECT (BASES + PIÈGES)

async dans useEffect

✓ *pattern recommandé :*

```
useEffect(() => {  
  const load = async () => {  
    const data = await fetchData();  
    setItems(data);  
  };  
  load();  
}, []);
```

Gestion loading

```
const [loading, setLoading] = useState(true);  
  
useEffect(() => {  
  const load = async () => {  
    setLoading(true);  
    const data = await fetchData();  
    setItems(data);  
    setLoading(false);  
  };  
  load();  
}, []);
```

Gestion error

```
const [error, setError] = useState("");  
  
try { ... } catch(e) { setError("Erreur de chargement"); }
```

Mini-exo + correction

Exo : afficher “Chargement...” puis la liste, sinon “Erreur”.

✓ *Correction (pattern)*

```
if (loading) return <p>Chargement...</p>;  
if (error) return <p>{error}</p>;  
return <List items={items} />;
```

PERSISTANCE LOCALSTORAGE (SANS BACKEND)

Objectifs

Sauvegarder une liste dans localStorage

Restaurer au rechargement

Comprendre quand sauvegarder (useEffect)

localStorage en 2 lignes

localStorage.setItem("key", valueString)

localStorage.getItem("key")

1 JSON obligatoire

```
localStorage.setItem("items", JSON.stringify(items));  
const saved = JSON.parse(localStorage.getItem("items") || "[]");
```

2 Charger depuis localStorage au montage

```
useEffect(() => {  
  const saved = JSON.parse(localStorage.getItem("items") || "[]");  
  setItems(saved);  
}, []);
```

3 Sauvegarder quand items change

```
useEffect(() => {  
  localStorage.setItem("items", JSON.stringify(items));  
}, [items]);
```

Bonnes pratiques

clé stable (ex: "formations")

valeurs JSON

fallback || "[]"

Mini-exo + correction

Exo : persister un CRUD local (les éléments restent après refresh)

✅ Correction = combinaison 2 + 3

PERSISTANCE LOCALSTORAGE (SANS BACKEND)

Bonus : custom hook useLocalStorage

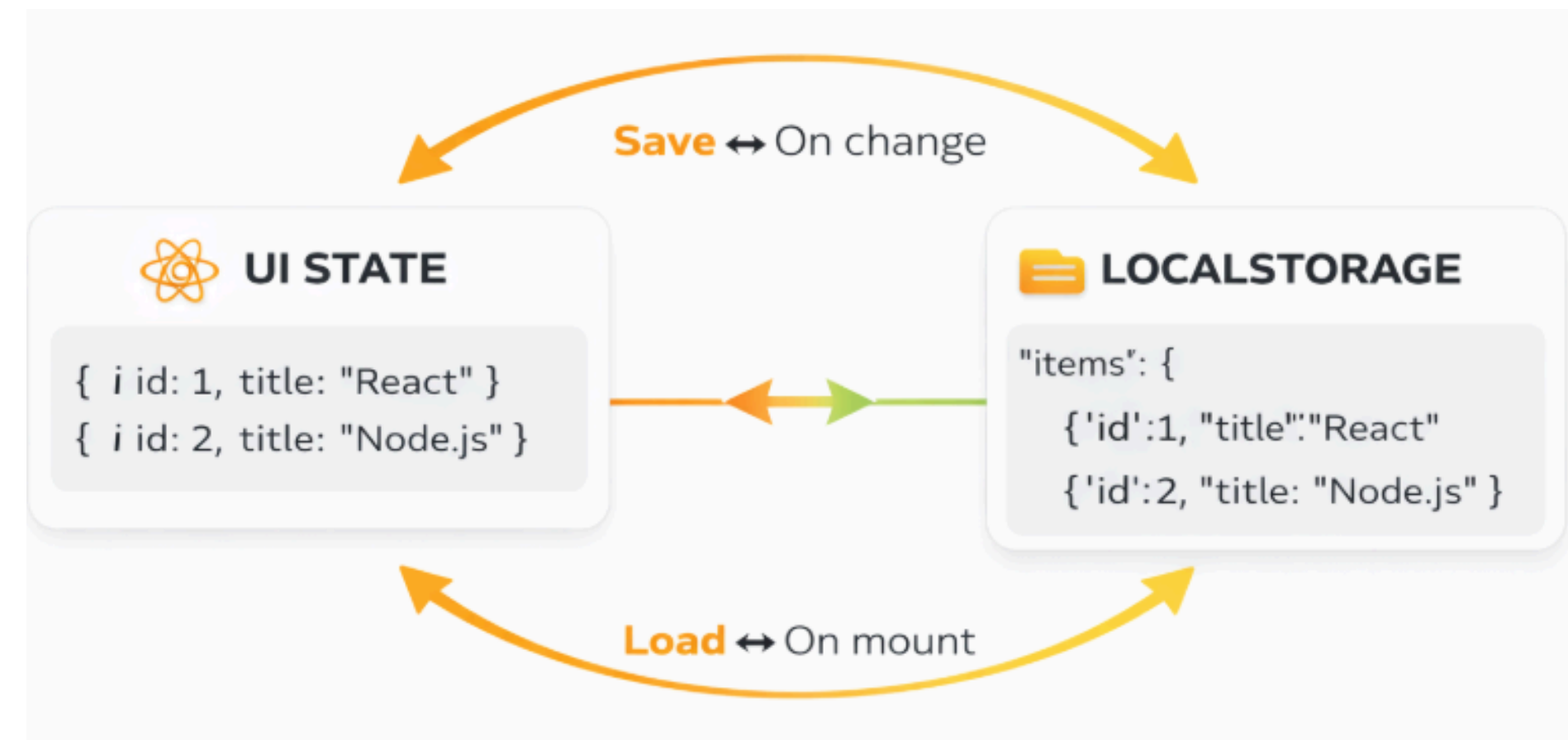
```
function useLocalStorageState(key, initialValue) {  
  const [value, setValue] = useState(() => {  
    const saved = localStorage.getItem(key);  
    return saved ? JSON.parse(saved) : initialValue;  
  });  
  
  useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);  
  
  return [value, setValue];  
}
```

Utilisation du hook

```
const [formations, setFormations] = useLocalStorageState("formations", []);
```

TP court

- Ajouter persistance à ton CRUD existant
- tester refresh
- vérifier qu'aucune erreur JSON n'apparait



FAKE API + SERVICES + ARCHITECTURE

On simule un appel réseau : on obtient une Promise + un délai + un résultat.

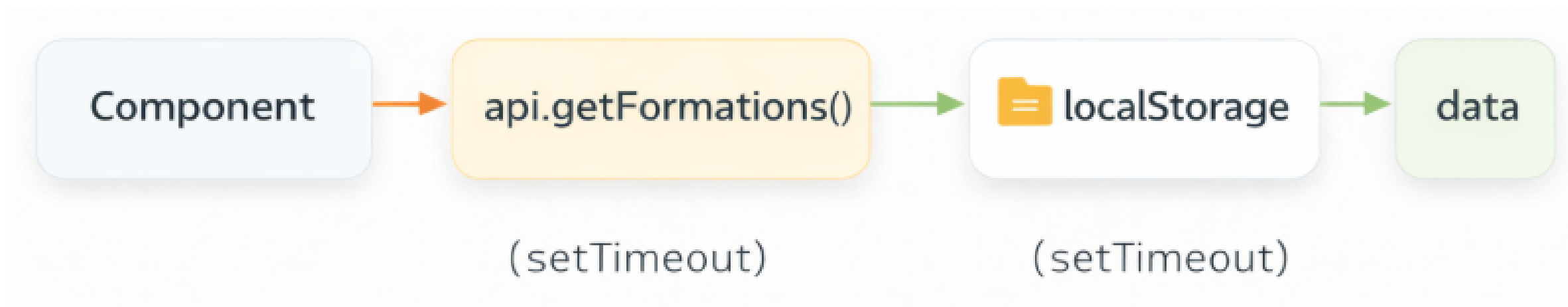
Pourquoi on fait ça ?

- apprendre à coder comme en entreprise (loading / error / async)
- préparer la transition vers une vraie API plus tard
- éviter d'avoir un backend au début

Exemple mental

■ Vraie API : `fetch("/formations")` → attend → reçoit JSON

■ Fake API : `api.getFormations()` → attend → reçoit JSON (depuis `localStorage`)



FAKE API + SERVICES + ARCHITECTURE

SERVICE LAYER – Le dossier `services/`

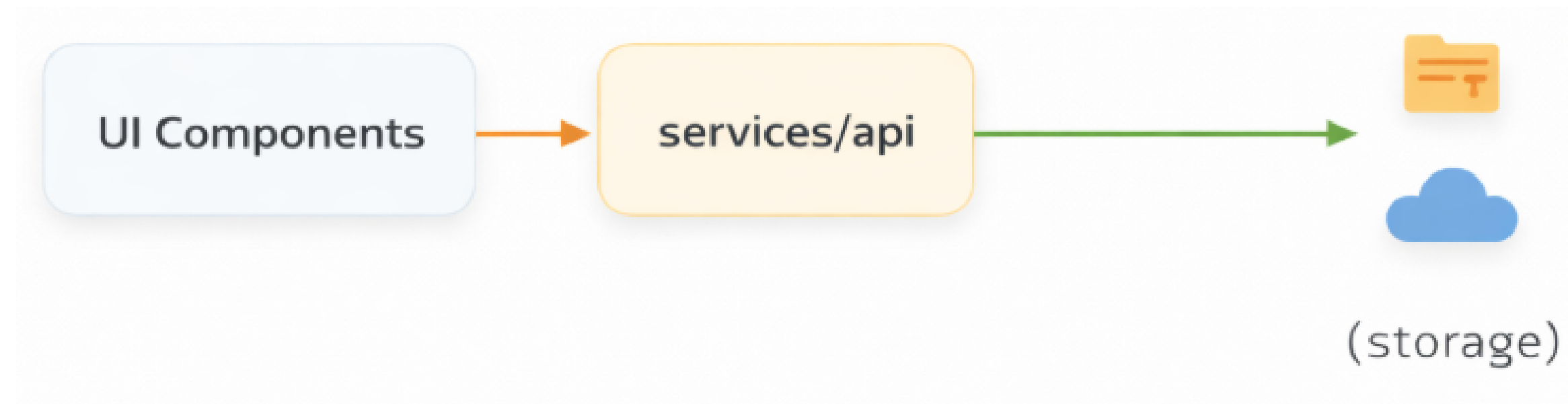
Les composants ne doivent pas gérer où sont les données (localStorage, API, etc.)

Objectif

- *éviter du code `localStorage` partout*
- *rendre les composants plus simples*
- *changer plus tard vers un vrai backend sans tout casser*

Règle

- ✓ Le composant appelle api
- ✗ Le composant ne touche pas directement `localStorage`



FAKE API + SERVICES + ARCHITECTURE

Fake API – GET (récupérer des formations)

services/fakeApi.js

```
export const api = {
  getFormations: () =>
    new Promise((resolve) => {
      setTimeout(() => {
        const data = JSON.parse(localStorage.getItem("formations") || "[]");
        resolve(data);
      }, 600);
    })
};
```

✓ Ce que ça fait, étape par étape :

- retourne une Promise (comme un fetch)
- attend 600ms (latence réseau)
- lit localStorage
- renvoie la liste

⚠ Point important :

- si la clé n'existe pas → [] (fallback)

FAKE API + SERVICES + ARCHITECTURE

Utilisation dans un composant (useEffect + loading + error)

Appeler la Fake API dans un composant

```
import { useEffect, useState } from "react";
import { api } from "../services/fakeApi";

export default function FormationsPage() {
  const [formations, setFormations] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const load = async () => {
      try {
        setLoading(true);
        const data = await api.getFormations();
        setFormations(data);
      } catch (e) {
        setError("Erreur de chargement");
      } finally {
        setLoading(false);
      }
    };
    load();
  }, []);

  if (loading) return <p>Chargement...</p>;
  if (error) return <p style={{ color: "red" }}>{error}</p>;
  return <pre>{JSON.stringify(formations, null, 2)}</pre>;
}
```

✓ Pourquoi `useEffect([])` ?

on charge les données au montage, comme une vraie page.

FAKE API + SERVICES + ARCHITECTURE

Fake API – CREATE (ajouter une formation)

```
export const api = {
  createFormation: (formation) =>
    new Promise((resolve) => {
      setTimeout(() => {
        const list = JSON.parse(localStorage.getItem("formations") || "[]");
        const newItem = { ...formation, id: Date.now() };
        const updated = [...list, newItem];
        localStorage.setItem("formations", JSON.stringify(updated));
        resolve(newItem);
      }, 600);
    })
};
```

✓ Résumé :

- lit list
- crée newItem
- sauvegarde updated
- renvoie newItem

FAKE API + SERVICES + ARCHITECTURE

Exo + Correction

Dans ton formulaire :

- appeler `api.createFormation(form)`
- puis mettre à jour l'UI

Correction simple (sans reload)

```
const handleAdd = async (form) => {  
  setLoading(true);  
  const newItem = await api.createFormation(form);  
  setFormations(prev => [...prev, newItem]);  
  setLoading(false);  
};
```

✅ *Avantage : UI se met à jour sans recharger toute la liste.*

CONTEXT API + AUTH MOCK + RÔLES + PLANNING (100% FRONT)

Objectif

- Introduire state global (Context)
- Auth simple (admin/étudiant)
- Routes protégées selon rôle
- Planning : créneaux + propositions + acceptation/refus

Context API (problème + solution)

Objectifs

- Comprendre “prop drilling”
- Mettre en place AuthContext
- Utiliser useContext

Problème : prop drilling (illustration)

Parent → child → grandchild → ... pour passer user

Solution : Context

- un Provider
- n'importe quel composant peut lire user

Création du context

```
import { createContext } from "react";  
export const AuthContext = createContext(null);
```

Provider

```
export function AuthProvider({ children }) {  
  const [user, setUser] = useState(null);  
  return (  
    <AuthContext.Provider value={{ user, setUser }}>  
      {children}  
    </AuthContext.Provider>  
  );  
}
```

CONTEXT API + AUTH MOCK + RÔLES + PLANNING (100% FRONT)

Utilisation

```
const { user, setUser } = useContext(AuthContext);
```

Login mock

```
setUser({ id: 1, name: "Admin", role: "ADMIN" });
```

Logout

```
setUser(null);
```

Mini-exo

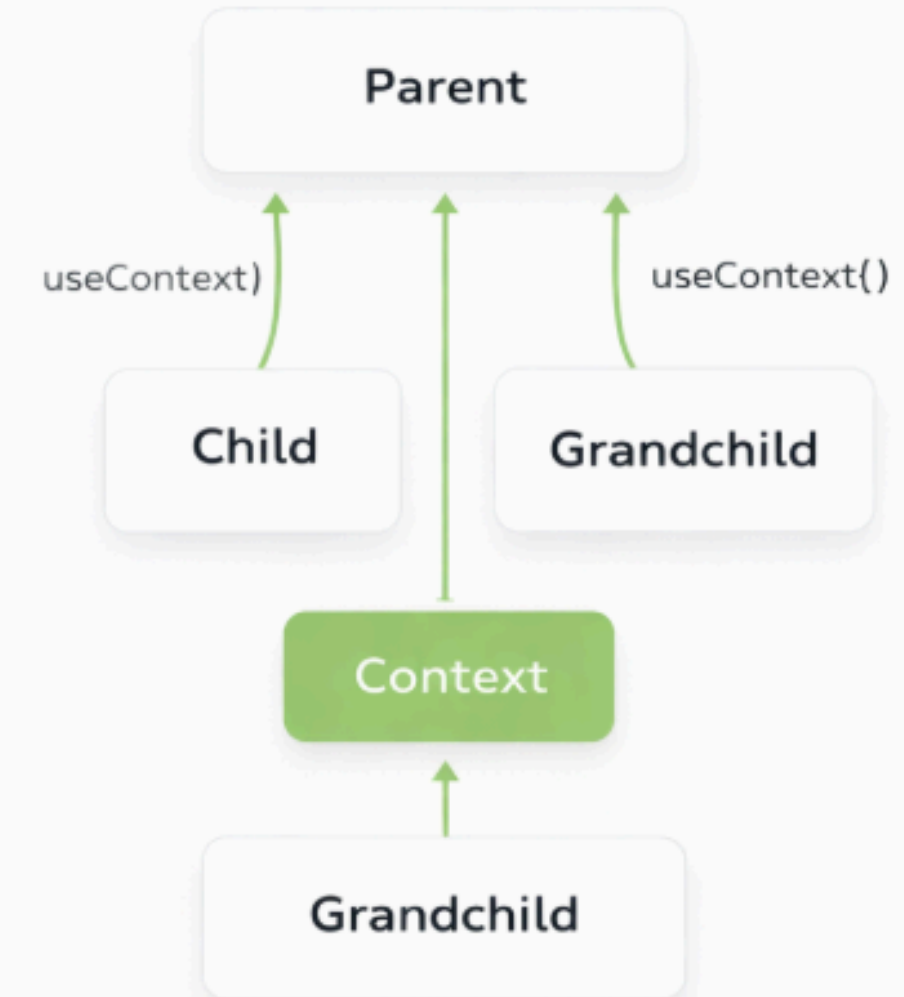
Exo : afficher "Bonjour {name}" ou "Non connecté"

✅ Correction : condition + useContext

PROP DRILLING



CONTEXT

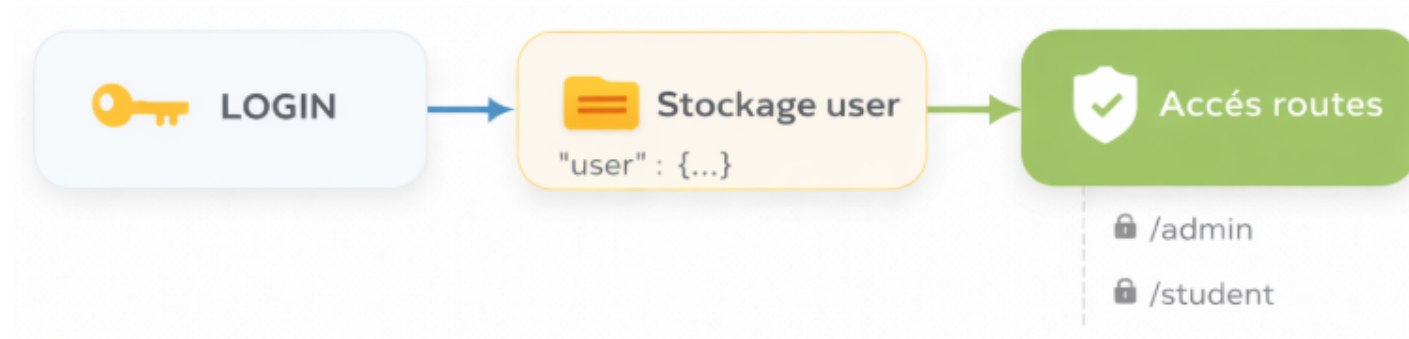


CONTEXT API + AUTH MOCK + RÔLES + PLANNING (100% FRONT)

Routes protégées (rôle)

Objectifs

- protéger /admin
- protéger /student
- rediriger selon rôle



ProtectedRoute

```
import { Navigate } from "react-router-dom";
import { useContext } from "react";
import { AuthContext } from "../AuthContext";

export function ProtectedRoute({ role, children }) {
  const { user } = useContext(AuthContext);
  if (!user) return <Navigate to="/login" replace />;
  if (role && user.role !== role) return <Navigate to="/" replace />;
  return children;
}
```

Utilisation routes

```
<Route path="/admin" element={
  <ProtectedRoute role="ADMIN"><AdminDashboard /></ProtectedRoute>
} />
```

Navigation conditionnelle

Afficher menu admin OU menu étudiant selon user.role

Exo : bouton "Espace admin" visible seulement pour admin

✓ **correction** : {user?.role === "ADMIN" && (...)}

- Persister session (localStorage)
- Au login : save user. Au mount : restore user.Code persistance

```
localStorage.setItem("user", JSON.stringify(user));
```

Restore

```
useEffect(() => {
  const saved = localStorage.getItem("user");
  if (saved) setUser(JSON.parse(saved));
}, []);
```

Pièges

- user null au premier render → gérer proprement
- redirections infinies → vérifier routes

CONTEXT API + AUTH MOCK + RÔLES + PLANNING (100% FRONT)

Planning + proposition/acceptation (100% front)

Objectifs

- gérer des créneaux de disponibilité
- proposer une séance
- étudiant accepte ou propose autre créneau
- afficher planning

Slide 2 – Modèle de données (simple)

```
slot: { id, day, start, end }  
session: { id, slotId, studentId, status: "PROPOSED" | "ACCEPTED" | "COUNTER" }
```

CRUD Slots (admin)

ajouter un slot

supprimer un slot

Slide 4 – Proposer une séance

Admin choisit :

élève

slot

→ crée session “PROPOSED”

Slide 5 – Vue étudiant

liste des séances proposées

bouton “Accepter”

bouton “Proposer autre créneau”

State update (pattern)

```
setSessions(sessions.map(s => s.id === id ? { ...s, status: "ACCEPTED" } : s));
```

Proposer autre créneau (COUNTER)

Étudiant choisit un autre slot → status “COUNTER”

Slide 8 – Mini-exo + correction

Exo : créer un bouton “Accepter” qui change le status

✓ correction : slide 6

Slide 9 – UI planning (simple)

tableau par jour

cards de sessions

Slide 10 – TP final séance 4

TP : finaliser app :

login admin/étudiant

routes protégées

slots + sessions + acceptation

Visuel + prompt

mini calendar weekly view + cards “Proposé / Accepté / Contre-proposé”

Merci pour votre attention